

Untitled21

October 10, 2024

```
[ ]: \begin{document}
#Program Name: IST 692 Lab 1
#Program Author: K.Roller
#Program Created: October 9th, 2024
#Program Purpose: To perform outlined task in lab 1
#Reason Lab is in jupyter instead of collab:
#Unable to succesfully run logistic regression in collab notebook. Attempted to
    ↪change the max iterations to
# various values ranging from 10000 to 10000000 with no success, I kept getting
    ↪a failure to converge error.
# I also attempted to specify a "solver" in the logistic regression and this
    ↪also made no difference.
# Lastly I reran all the code several times thinking I may have missed a block
    ↪with no success.
# However, as soon as I copied the code into this notebook it started working.
# At this time I do not have an answer for why this is, Though I apologize for
    ↪turning in this format instead
```

```
[2]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Load the dataset from UCI (it's available via many sources or from your own
    ↪directory)
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.
    ↪data"
column_names = ['age', 'workclass', 'fnlwgt', 'education', 'education_num',
    ↪'marital_status',
                'occupation', 'relationship', 'race', 'sex', 'capital_gain',
    ↪'capital_loss',
                'hours_per_week', 'native_country', 'income']

# Load the dataset into a pandas DataFrame
df = pd.read_csv(url, names=column_names, na_values=' ?')

# Display the first few rows of the dataset
```

```
df.head()
```

```
[2]:
```

	age	workclass	fnlwgt	education	education_num	\
0	39	State-gov	77516	Bachelors	13	
1	50	Self-emp-not-inc	83311	Bachelors	13	
2	38	Private	215646	HS-grad	9	
3	53	Private	234721	11th	7	
4	28	Private	338409	Bachelors	13	

	marital_status	occupation	relationship	race	sex	\
0	Never-married	Adm-clerical	Not-in-family	White	Male	
1	Married-civ-spouse	Exec-managerial	Husband	White	Male	
2	Divorced	Handlers-cleaners	Not-in-family	White	Male	
3	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	
4	Married-civ-spouse	Prof-specialty	Wife	Black	Female	

	capital_gain	capital_loss	hours_per_week	native_country	income
0	2174	0	40	United-States	<=50K
1	0	0	13	United-States	<=50K
2	0	0	40	United-States	<=50K
3	0	0	40	United-States	<=50K
4	0	0	40	Cuba	<=50K

```
[3]: df.columns
```

```
[3]: Index(['age', 'workclass', 'fnlwgt', 'education', 'education_num',  
         'marital_status', 'occupation', 'relationship', 'race', 'sex',  
         'capital_gain', 'capital_loss', 'hours_per_week', 'native_country',  
         'income'],  
         dtype='object')
```

```
[4]: len(df)
```

```
[4]: 32561
```

```
[5]: # Drop rows with missing values  
df = df.dropna()  
  
# Get basic descriptive statistics  
df.describe()
```

```
[5]:
```

	age	fnlwgt	education_num	capital_gain	capital_loss	\
count	30162.000000	3.016200e+04	30162.000000	30162.000000	30162.000000	
mean	38.437902	1.897938e+05	10.121312	1092.007858	88.372489	
std	13.134665	1.056530e+05	2.549995	7406.346497	404.298370	
min	17.000000	1.376900e+04	1.000000	0.000000	0.000000	
25%	28.000000	1.176272e+05	9.000000	0.000000	0.000000	
50%	37.000000	1.784250e+05	10.000000	0.000000	0.000000	

75%	47.000000	2.376285e+05	13.000000	0.000000	0.000000
max	90.000000	1.484705e+06	16.000000	99999.000000	4356.000000

	hours_per_week
count	30162.000000
mean	40.931238
std	11.979984
min	1.000000
25%	40.000000
50%	40.000000
75%	45.000000
max	99.000000

```
[6]: len(df)
```

```
[6]: 30162
```

```
[7]: # Categorical feature distributions
categorical_columns = ['relationship', 'native_country', 'race', 'sex', 'education', 'workclass', 'marital_status', 'occupation', 'income']

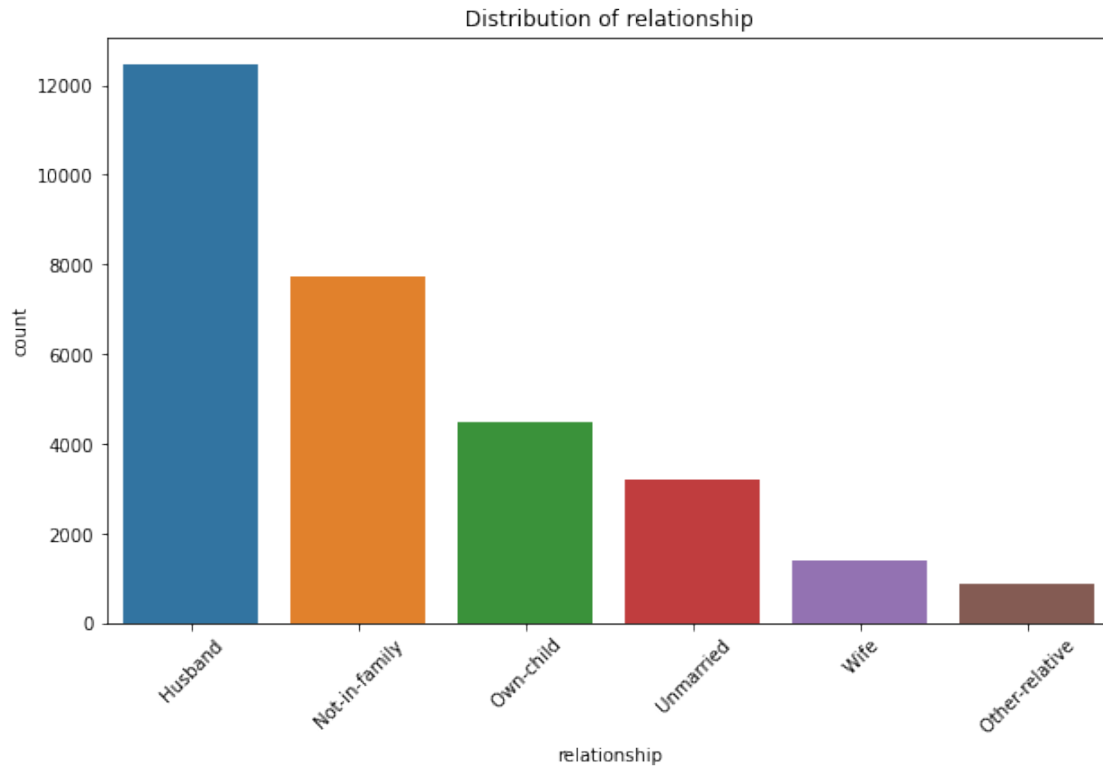
# Displaying the distribution of each categorical variable
for col in categorical_columns:
    print(f"\nDistribution of {col}:\n")
    print(df[col].value_counts(normalize=True) * 100)

    # Plot each categorical variable in a separate chart
    plt.figure(figsize=(10, 6))
    sns.countplot(data=df, x=col, order=df[col].value_counts().index)
    plt.title(f'Distribution of {col}')
    plt.xticks(rotation=45) # Rotate x-axis labels for readability
    plt.show()
```

Distribution of relationship:

Husband	41.320204
Not-in-family	25.615012
Own-child	14.806710
Unmarried	10.649161
Wife	4.661495
Other-relative	2.947417

Name: relationship, dtype: float64

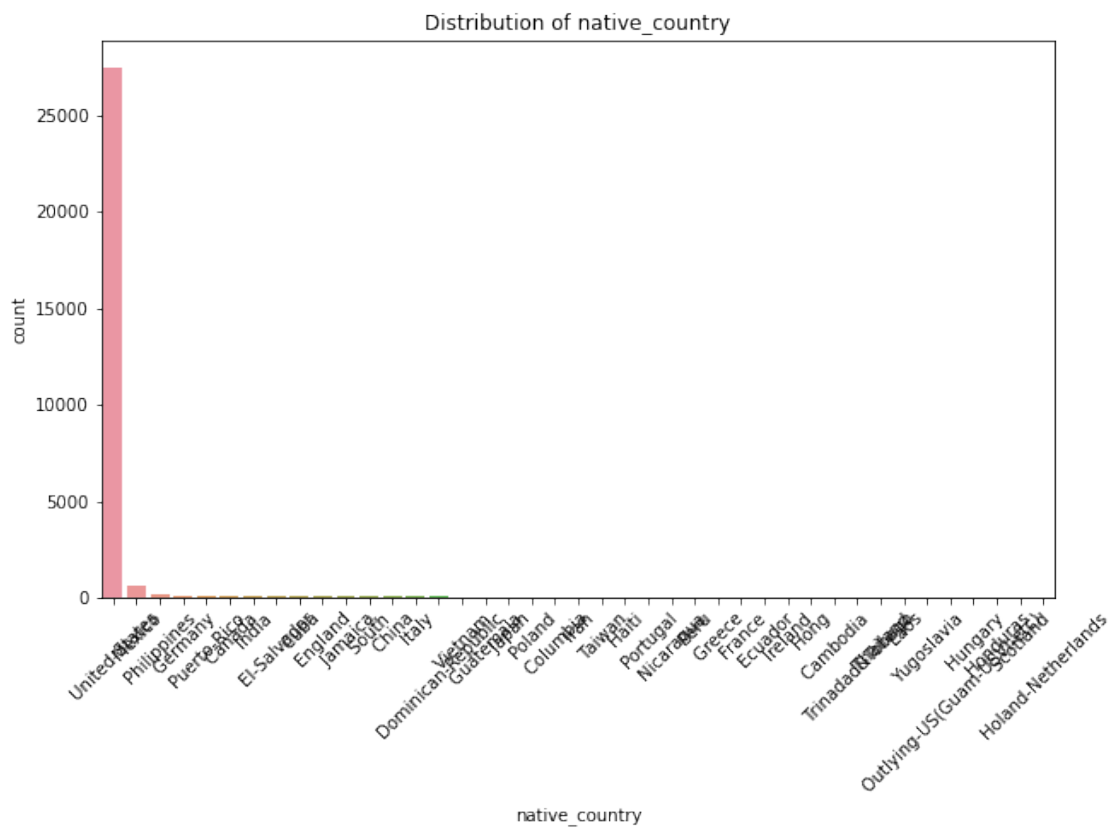


Distribution of native_country:

United-States	91.187587
Mexico	2.022412
Philippines	0.623301
Germany	0.424375
Puerto-Rico	0.361382
Canada	0.354751
India	0.331543
El-Salvador	0.331543
Cuba	0.305020
England	0.285127
Jamaica	0.265234
South	0.235396
China	0.225449
Italy	0.225449
Dominican-Republic	0.222134
Vietnam	0.212188
Guatemala	0.208872
Japan	0.195610
Poland	0.185664
Columbia	0.185664

Iran	0.139248
Taiwan	0.139248
Haiti	0.139248
Portugal	0.112725
Nicaragua	0.109409
Peru	0.099463
Greece	0.096147
France	0.089517
Ecuador	0.089517
Ireland	0.079570
Hong	0.062993
Cambodia	0.059678
Trinidad&Tobago	0.059678
Thailand	0.056362
Laos	0.056362
Yugoslavia	0.053047
Outlying-US(Guam-USVI-etc)	0.046416
Hungary	0.043101
Honduras	0.039785
Scotland	0.036470
Holand-Netherlands	0.003315

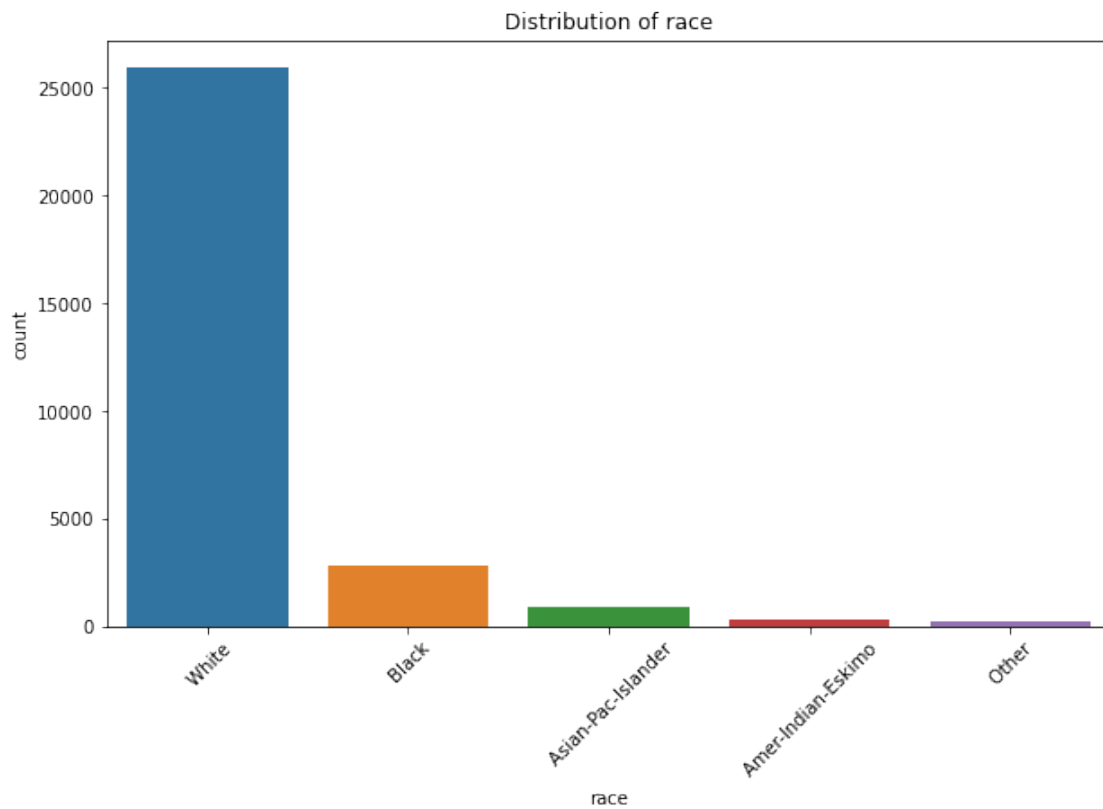
Name: native_country, dtype: float64



Distribution of race:

White	85.979046
Black	9.339566
Asian-Pac-Islander	2.967310
Amer-Indian-Eskimo	0.948213
Other	0.765864

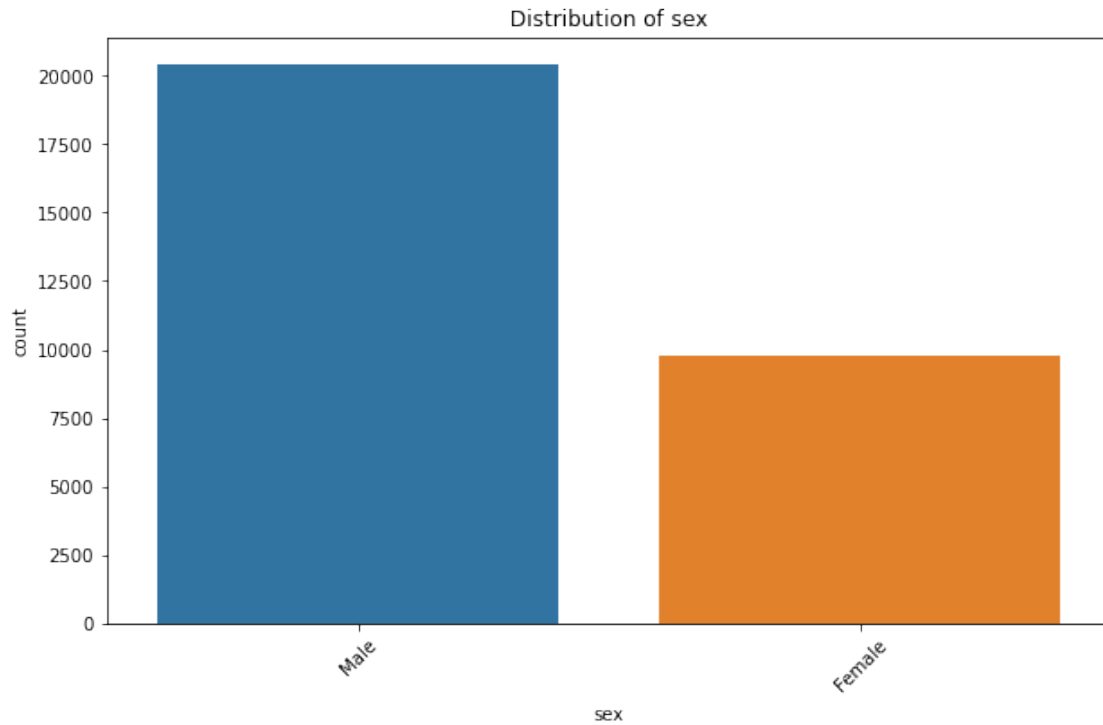
Name: race, dtype: float64



Distribution of sex:

Male	67.568464
Female	32.431536

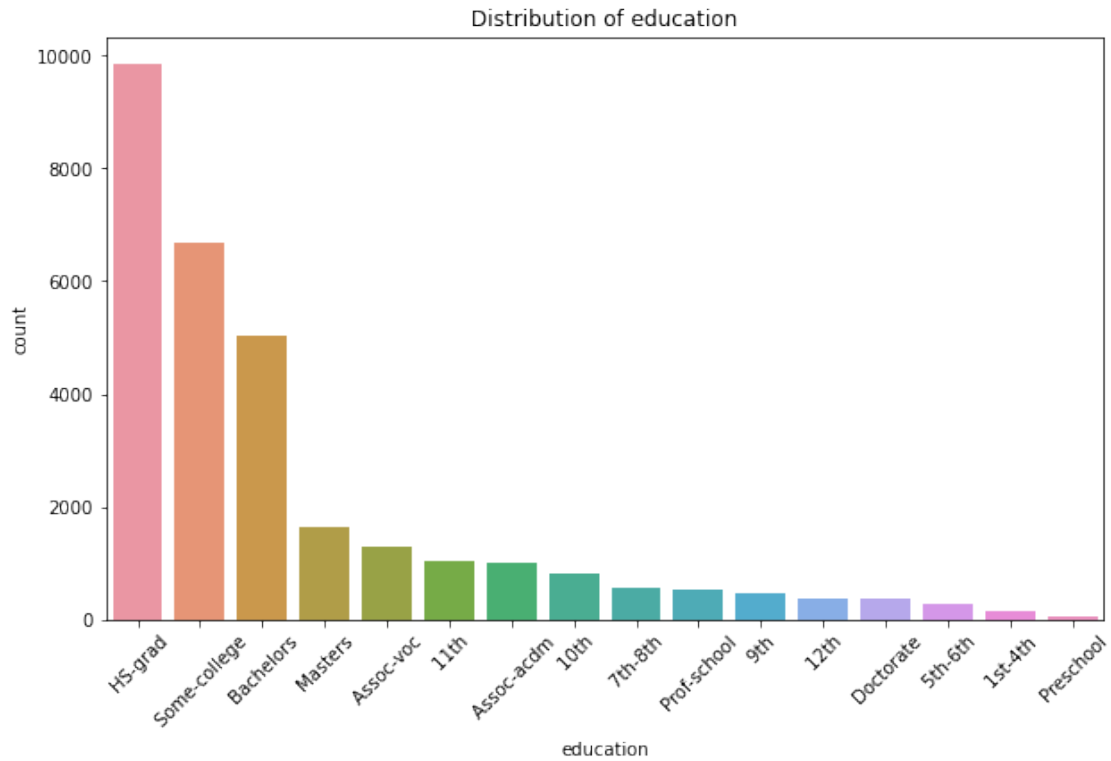
Name: sex, dtype: float64



Distribution of education:

HS-grad	32.623831
Some-college	22.140442
Bachelors	16.723029
Masters	5.394205
Assoc-voc	4.333267
11th	3.474571
Assoc-acdm	3.341953
10th	2.718653
7th-8th	1.846695
Prof-school	1.796963
9th	1.508521
12th	1.249917
Doctorate	1.243286
5th-6th	0.954844
1st-4th	0.500630
Preschool	0.149194

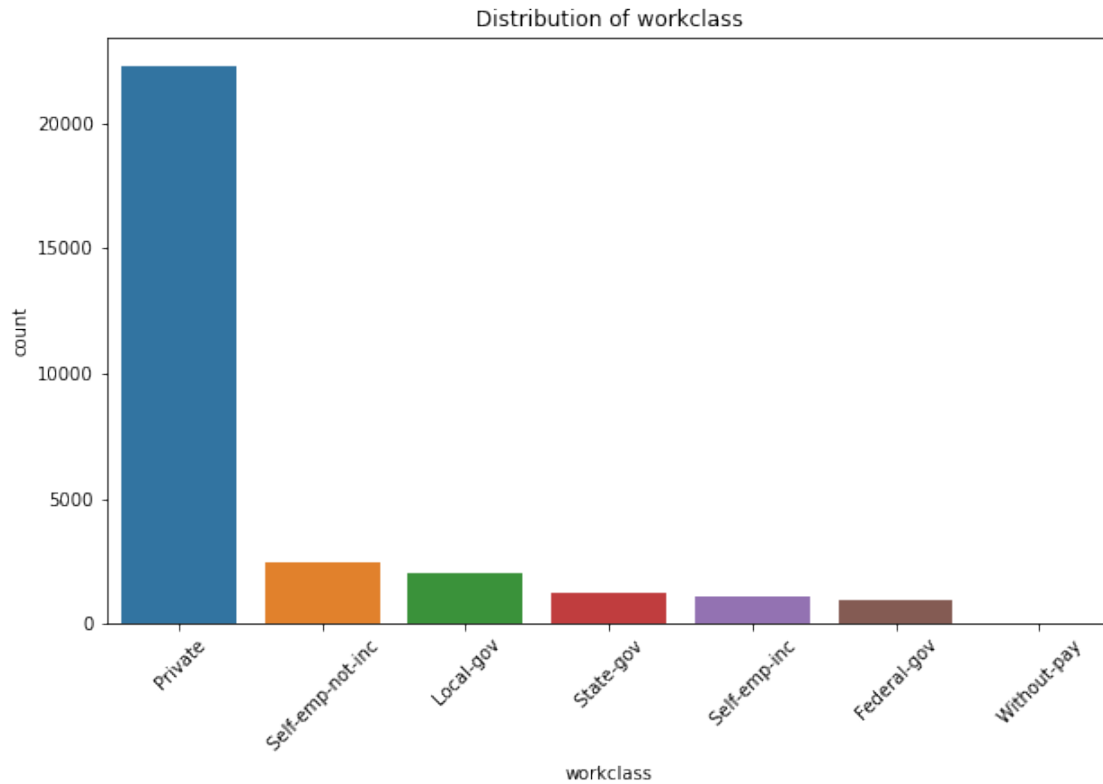
Name: education, dtype: float64



Distribution of workclass:

Private	73.887673
Self-emp-not-inc	8.285260
Local-gov	6.852994
State-gov	4.240435
Self-emp-inc	3.560772
Federal-gov	3.126451
Without-pay	0.046416

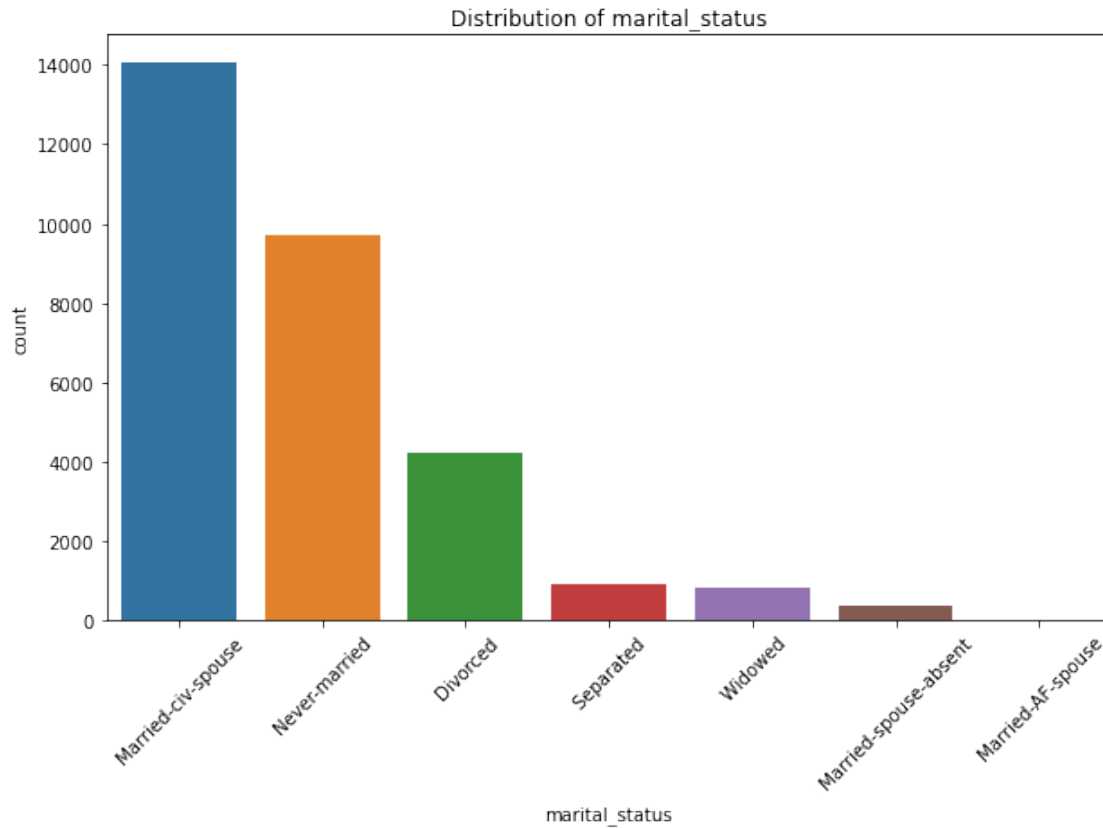
Name: workclass, dtype: float64



Distribution of marital_status:

Married-civ-spouse	46.631523
Never-married	32.245872
Divorced	13.971222
Separated	3.113189
Widowed	2.741861
Married-spouse-absent	1.226709
Married-AF-spouse	0.069624

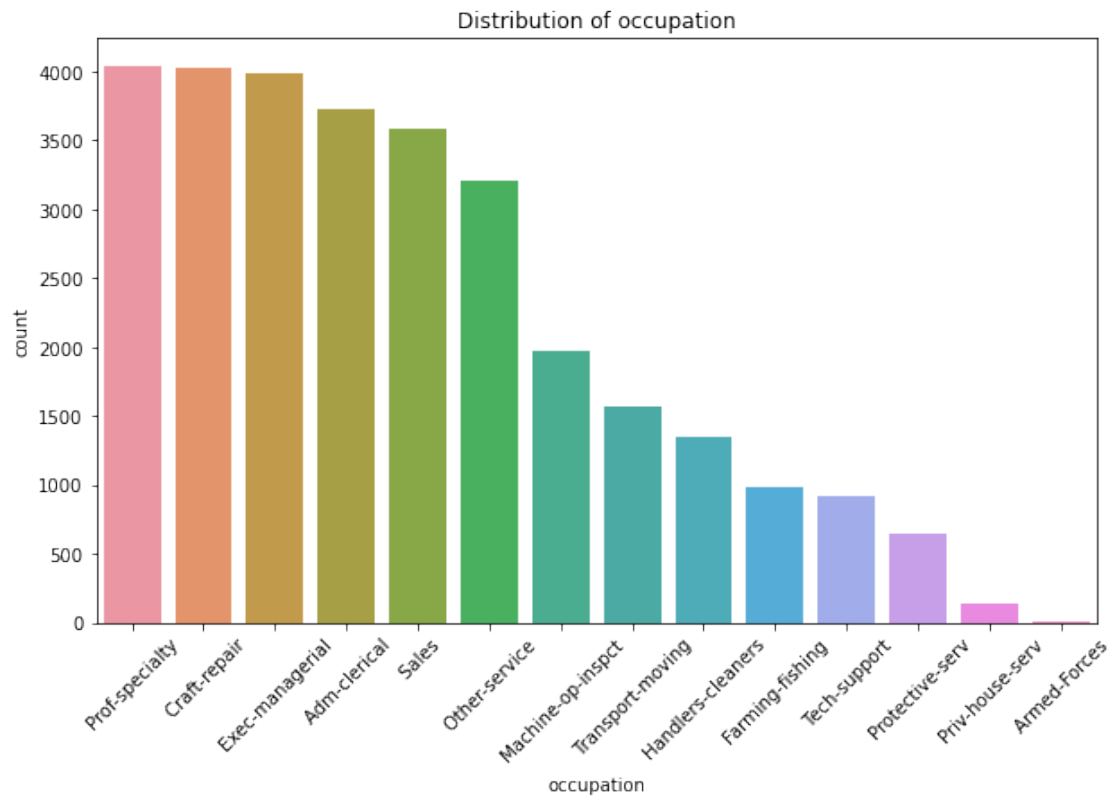
Name: marital_status, dtype: float64



Distribution of occupation:

Prof-specialty	13.387706
Craft-repair	13.361183
Exec-managerial	13.235197
Adm-clerical	12.336715
Sales	11.882501
Other-service	10.649161
Machine-op-inspct	6.518135
Transport-moving	5.211856
Handlers-cleaners	4.475831
Farming-fishing	3.278960
Tech-support	3.023672
Protective-serv	2.135137
Priv-house-serv	0.474106
Armed-Forces	0.029839

Name: occupation, dtype: float64

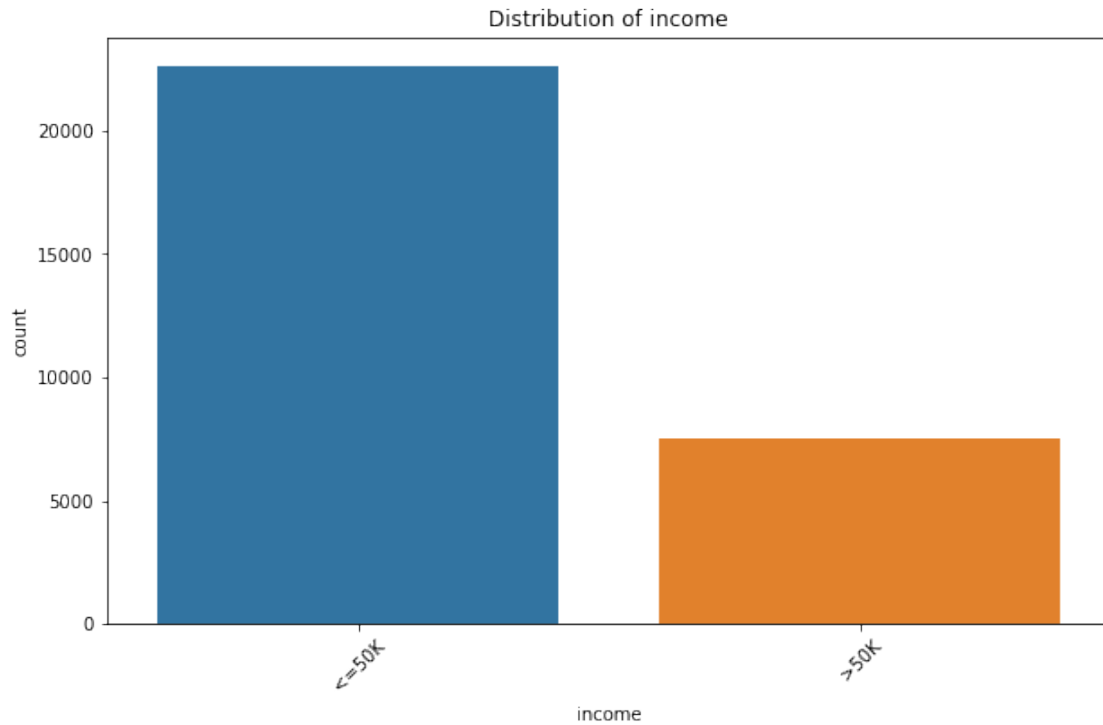


Distribution of income:

<=50K 75.107751

>50K 24.892249

Name: income, dtype: float64



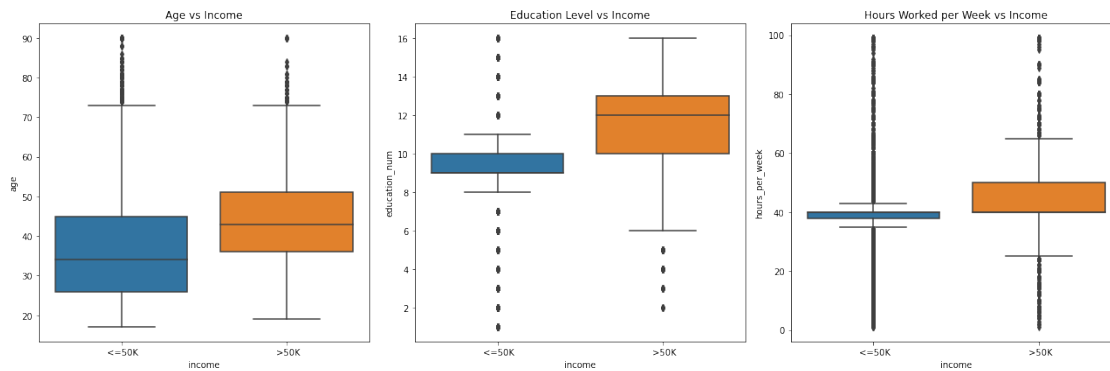
```
[8]: # Plotting age, education_num, and hours_per_week vs income
fig, ax = plt.subplots(1, 3, figsize=(18, 6))

sns.boxplot(data=df, x='income', y='age', ax=ax[0])
ax[0].set_title('Age vs Income')

sns.boxplot(data=df, x='income', y='education_num', ax=ax[1])
ax[1].set_title('Education Level vs Income')

sns.boxplot(data=df, x='income', y='hours_per_week', ax=ax[2])
ax[2].set_title('Hours Worked per Week vs Income')

plt.tight_layout()
plt.show()
```



```
[9]: import numpy as np

# Perform one-hot encoding for all categorical columns
df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)

# Check the column names to ensure 'income' is properly encoded
print(df_encoded.columns) # Look for the new income column name (likely
    ↳ 'income_>50K')

# Select only numeric columns for the correlation matrix
numeric_df_encoded = df_encoded.select_dtypes(include=[np.number])

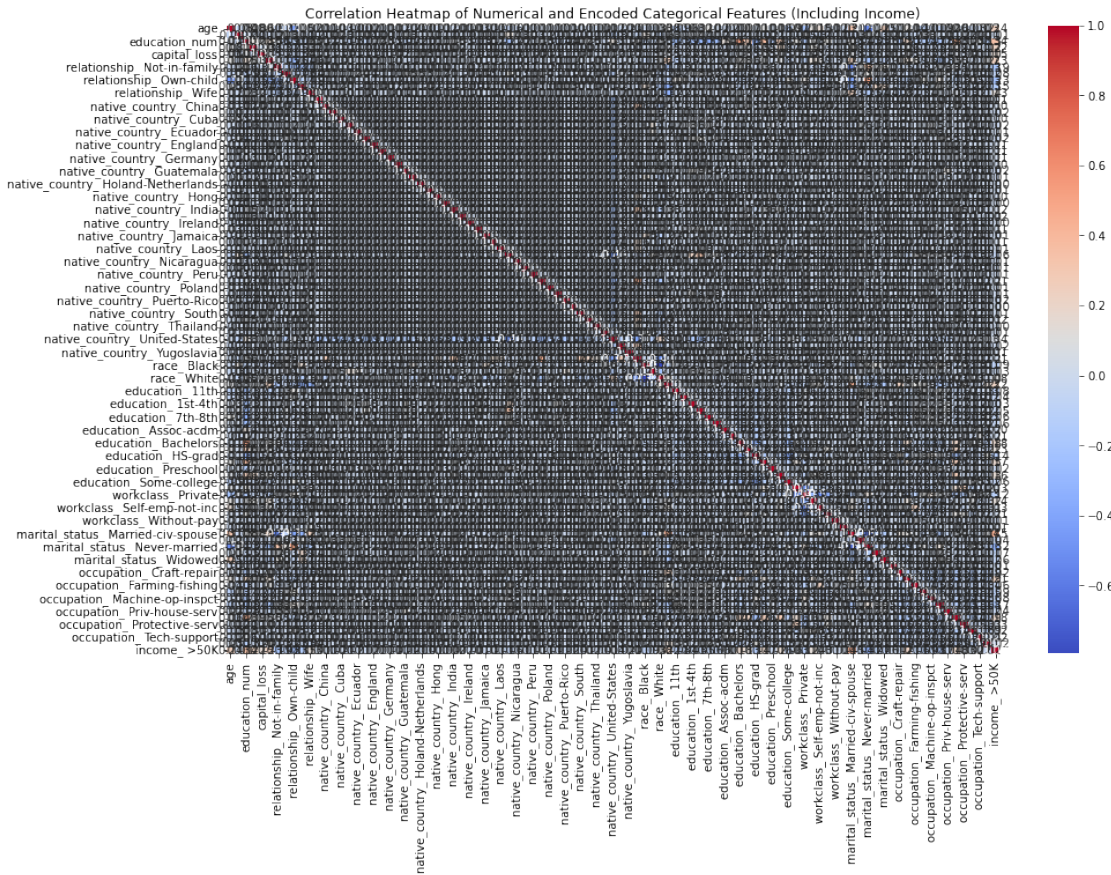
# Display the correlation heatmap, including income
plt.figure(figsize=(15, 10))
sns.heatmap(numeric_df_encoded.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title('Correlation Heatmap of Numerical and Encoded Categorical Features_
    ↳ (Including Income)')
plt.show()
#Note, this plot is much messier and more difficult to read versus the one_
    ↳ previously generated
# This is also a different plot, which should also be noted, as this is not_
    ↳ just a correlation matrix
# or heatmap of the encoded variables but seemingly every variable.
# This seems to be an error in the way the columns are specified for the_
    ↳ heatmap rather than an
# issue with the encoding of the variables. Just noting so you know that I know_
    ↳ this isnt the same thing.
```

```
Index(['age', 'fnlwgt', 'education_num', 'capital_gain', 'capital_loss',
      'hours_per_week', 'relationship_ Not-in-family',
      'relationship_ Other-relative', 'relationship_ Own-child',
      'relationship_ Unmarried', 'relationship_ Wife',
      'native_country_ Canada', 'native_country_ China',
      'native_country_ Columbia', 'native_country_ Cuba',
```

```

'native_country_ Dominican-Republic', 'native_country_ Ecuador',
'native_country_ El-Salvador', 'native_country_ England',
'native_country_ France', 'native_country_ Germany',
'native_country_ Greece', 'native_country_ Guatemala',
'native_country_ Haiti', 'native_country_ Holand-Netherlands',
'native_country_ Honduras', 'native_country_ Hong',
'native_country_ Hungary', 'native_country_ India',
'native_country_ Iran', 'native_country_ Ireland',
'native_country_ Italy', 'native_country_ Jamaica',
'native_country_ Japan', 'native_country_ Laos',
'native_country_ Mexico', 'native_country_ Nicaragua',
'native_country_ Outlying-US(Guam-USVI-etc)', 'native_country_ Peru',
'native_country_ Philippines', 'native_country_ Poland',
'native_country_ Portugal', 'native_country_ Puerto-Rico',
'native_country_ Scotland', 'native_country_ South',
'native_country_ Taiwan', 'native_country_ Thailand',
'native_country_ Trinidad&Tobago', 'native_country_ United-States',
'native_country_ Vietnam', 'native_country_ Yugoslavia',
'race_ Asian-Pac-Islander', 'race_ Black', 'race_ Other', 'race_ White',
'sex_ Male', 'education_ 11th', 'education_ 12th', 'education_ 1st-4th',
'education_ 5th-6th', 'education_ 7th-8th', 'education_ 9th',
'education_ Assoc-acdm', 'education_ Assoc-voc', 'education_ Bachelors',
'education_ Doctorate', 'education_ HS-grad', 'education_ Masters',
'education_ Preschool', 'education_ Prof-school',
'education_ Some-college', 'workclass_ Local-gov', 'workclass_ Private',
'workclass_ Self-emp-inc', 'workclass_ Self-emp-not-inc',
'workclass_ State-gov', 'workclass_ Without-pay',
'marital_status_ Married-AF-spouse',
'marital_status_ Married-civ-spouse',
'marital_status_ Married-spouse-absent',
'marital_status_ Never-married', 'marital_status_ Separated',
'marital_status_ Widowed', 'occupation_ Armed-Forces',
'occupation_ Craft-repair', 'occupation_ Exec-managerial',
'occupation_ Farming-fishing', 'occupation_ Handlers-cleaners',
'occupation_ Machine-op-inspct', 'occupation_ Other-service',
'occupation_ Priv-house-serv', 'occupation_ Prof-specialty',
'occupation_ Protective-serv', 'occupation_ Sales',
'occupation_ Tech-support', 'occupation_ Transport-moving',
'income_ >50K'],
dtype='object')

```



```
[10]: df_encoded.dtypes
```

```
[10]: age                int64
      fnlwgt            int64
      education_num     int64
      capital_gain      int64
      capital_loss      int64
      ...
      occupation_ Protective-serv  uint8
      occupation_ Sales           uint8
      occupation_ Tech-support    uint8
      occupation_ Transport-moving uint8
      income_ >50K                uint8
      Length: 97, dtype: object
```

```
[11]: from sklearn.model_selection import train_test_split
```

```
# Separate features (X) and target (y)
```

```

X = df_encoded.drop(columns=['income_ >50K']) # Features (drop the encoded
↳ income column)
y = df_encoded['income_ >50K'] # Target (income)

# Perform train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳ random_state=42)

# Check the shape of the resulting datasets
print("Training features shape:", X_train.shape)
print("Testing features shape:", X_test.shape)
print("Training labels shape:", y_train.shape)
print("Testing labels shape:", y_test.shape)

```

```

Training features shape: (21113, 96)
Testing features shape: (9049, 96)
Training labels shape: (21113,)
Testing labels shape: (9049,)

```

```

[12]: from sklearn.linear_model import LogisticRegression

# Create the Logistic Regression model
model = LogisticRegression(max_iter=1000)

# Train the model on the training data
model.fit(X_train, y_train)

```

```

[12]: LogisticRegression(max_iter=1000)

```

```

[13]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Predict on the test set
y_pred = model.predict(X_test)

# Compute the confusion matrix and transpose it
cm = confusion_matrix(y_test, y_pred)

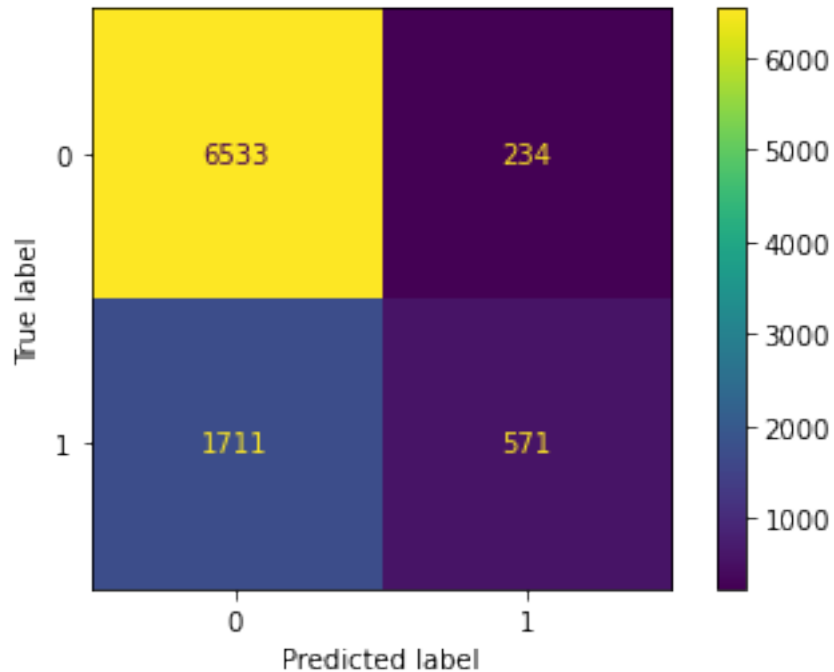
# Display the confusion matrix with transposed axes
ConfusionMatrixDisplay(confusion_matrix=cm).plot()

```

```

[13]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2c1c78b9af0>

```

```
[14]: from sklearn.metrics import precision_score, recall_score, f1_score,   
      ↪ accuracy_score
```

```
# Predict on the test set  
y_pred = model.predict(X_test)  
  
# Calculate Accuracy  
accuracy = accuracy_score(y_test, y_pred)  
print(f"Accuracy: {accuracy:.4f}")
```

Accuracy: 0.7851

```
[15]: # Calculate Precision  
precision = precision_score(y_test, y_pred)  
print(f"Precision: {precision:.4f}")
```

Precision: 0.7093

```
[16]: # Calculate Recall (Sensitivity or True Positive Rate)  
recall = recall_score(y_test, y_pred)  
print(f"Recall: {recall:.4f}")
```

Recall: 0.2502

```
[17]: # Calculate F1-Score  
f1 = f1_score(y_test, y_pred)
```

```
print(f"F1-Score: {f1:.4f}")
```

F1-Score: 0.3699

```
[18]: from sklearn.metrics import roc_curve, roc_auc_score

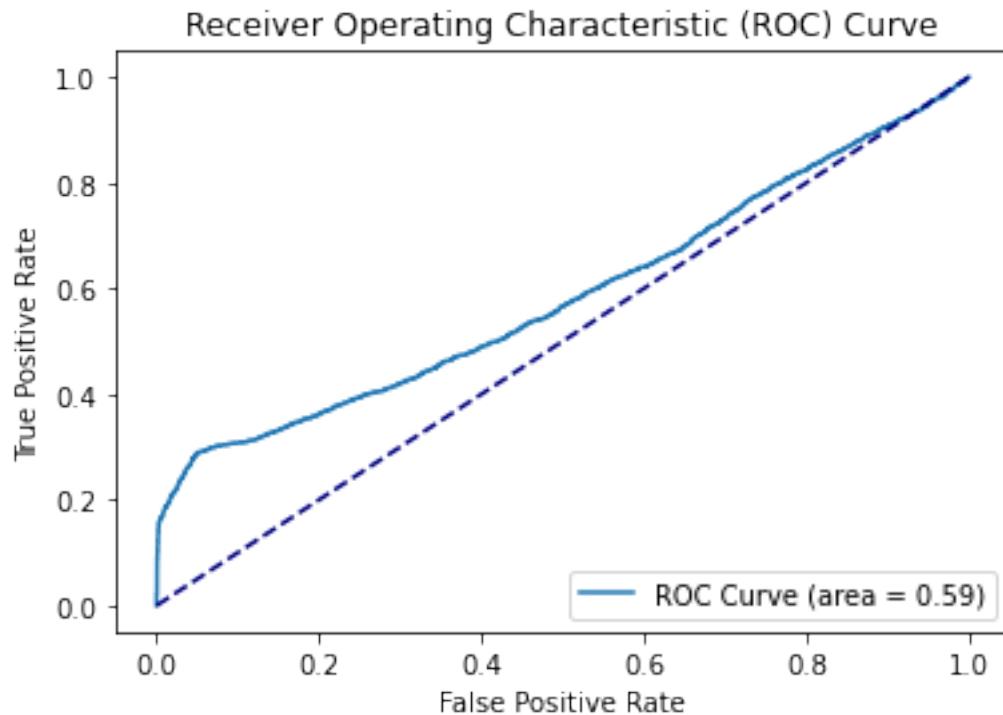
# Calculate the probability estimates for the test set
y_proba = model.predict_proba(X_test)[:, 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

# Compute AUC
auc = roc_auc_score(y_test, y_proba)

# Plot ROC Curve
plt.figure()
plt.plot(fpr, tpr, label=f"ROC Curve (area = {auc:.2f})")
plt.plot([0, 1], [0, 1], color="navy", linestyle="--")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()

# Print AUC score
print(f"AUC: {auc:.2f}")
```



AUC: 0.59

```
[19]: # Group the test data by race and gender to assess fairness
race_groups = X_test['race_ White']
sex_groups = X_test['sex_ Male']

# Calculate accuracy for different races and genders
accuracy_by_race = (y_pred[race_groups == 1] == y_test[race_groups == 1]).mean()
accuracy_by_sex = (y_pred[sex_groups == 1] == y_test[sex_groups == 1]).mean()

print(f"Accuracy for White individuals: {accuracy_by_race:.2f}")
print(f"Accuracy for Male individuals: {accuracy_by_sex:.2f}")
```

Accuracy for White individuals: 0.77

Accuracy for Male individuals: 0.74

```
[20]: race_black = X_test['race_ Black']
sex_female = X_test['sex_ Male'] == 0 # Assuming 1 = Male, 0 = Female

accuracy_by_race_black = (y_pred[race_black == 1] == y_test[race_black == 1]).
    ↪mean()

accuracy_by_sex_female = (y_pred[sex_female] == y_test[sex_female]).mean()
```

```
print(f"Accuracy for Black individuals: {accuracy_by_race_black:.2f}")
print(f"Accuracy for Female individuals: {accuracy_by_sex_female:.2f}")
```

Accuracy for Black individuals: 0.88
Accuracy for Female individuals: 0.89

```
[21]: from sklearn.ensemble import RandomForestClassifier
      from sklearn.metrics import accuracy_score

      # Train a Random Forest model
      rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
      rf_model.fit(X_train, y_train)

      # Predict on the test set
      y_pred_rf = rf_model.predict(X_test)

      # Calculate accuracy
      accuracy_rf = accuracy_score(y_test, y_pred_rf)
      print(f"Random Forest Accuracy: {accuracy_rf:.2f}")
```

Random Forest Accuracy: 0.85

```
[22]: from sklearn.ensemble import GradientBoostingClassifier
      from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

      # Initialize the Gradient Boosting model
      gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)

      # Train the model on the training data
      gb_model.fit(X_train, y_train)

      # Predict on the test set
      y_pred_gb = gb_model.predict(X_test)

      # Calculate accuracy, precision, recall, and F1 score
      accuracy_gb = accuracy_score(y_test, y_pred_gb)
      print(f"Gradient Boosting Accuracy: {accuracy_gb:.2f}")
```

Gradient Boosting Accuracy: 0.86

```
[23]: """Your Task:
      Plot and interpret the Confusion Matrices and ROC Curves for the two new models.
      Calculate the precision, recall, and F1 scores for both models.
      Test the models' performance on protected groups (e.g., race, gender) in the dataset.
```

Based on this evaluation, decide which model you would choose and explain your reasoning.
Although this dataset isn't classifying people into high-stakes categories like "offender" or "non-offender,"
it is predicting whether someone earns more than \$50k a year. From a responsible AI perspective,
this can still be problematic. Explain how this type of prediction could negatively impact the individuals
being classified by the algorithm, as well as the broader group of stakeholders who might not interact
with the algorithm directly but could still be affected by its outcomes."

[23]: 'Your Task:\nPlot and interpret the Confusion Matrices and ROC Curves for the two new models.\nCalculate the precision, recall, and F1 scores for both models.\nTest the models\' performance on protected groups (e.g., race, gender) in the dataset.\nBased on this evaluation, decide which model you would choose and explain your reasoning.\nAlthough this dataset isn\'t classifying people into high-stakes categories like "offender" or "non-offender," \nit is predicting whether someone earns more than \$50k a year. From a responsible AI perspective, \nthis can still be problematic. Explain how this type of prediction could negatively impact the individuals\nbeing classified by the algorithm, as well as the broader group of stakeholders who might not interact\nwith the algorithm directly but could still be affected by its outcomes.'

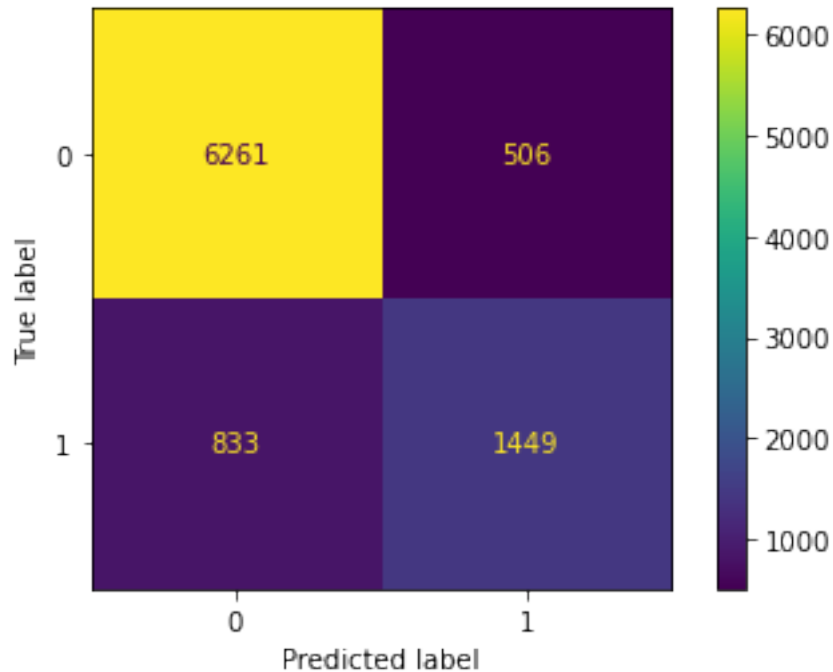
[24]: *#START ASSIGNMENT WORK*

```
#FOR RANDOM FOREST MODEL
# Predict on the test set
y_pred_rf = rf_model.predict(X_test)

# Compute the confusion matrix and transpose it
cm1 = confusion_matrix(y_test, y_pred_rf)

# Display the confusion matrix with transposed axes
ConfusionMatrixDisplay(confusion_matrix=cm1).plot()
```

[24]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2c1ced87b20>



```
[27]: # FOR RANDOM FOREST

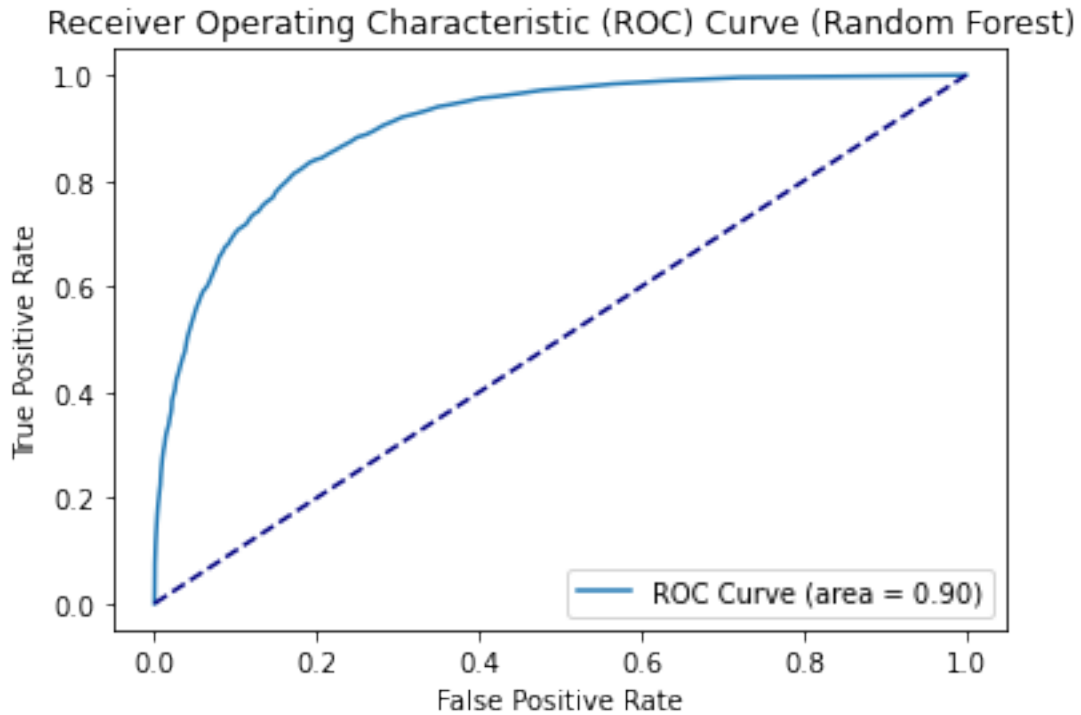
# Calculate the probability estimates for the test set
y_proba = rf_model.predict_proba(X_test)[: , 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

# Compute AUC
auc = roc_auc_score(y_test, y_proba)

# Plot ROC Curve
plt.figure()
plt.plot(fpr, tpr, label=f"ROC Curve (area = {auc:.2f})")
plt.plot([0, 1], [0, 1], color="navy", linestyle="--")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve (Random Forest)')
plt.legend(loc="lower right")
plt.show()

# Print AUC score
print(f"AUC: {auc:.2f}")
```



AUC: 0.90

```
[28]: #RANDOM FOREST

# Predict on the test set
y_pred = rf_model.predict(X_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred_rf)
print(f"Accuracy: {accuracy:.4f}")
```

Accuracy: 0.8520

```
[29]: #Random Forest

# Calculate Precision
precision = precision_score(y_test, y_pred_rf)
print(f"Precision: {precision:.4f}")
```

Precision: 0.7412

```
[30]: #Random Forest

# Calculate Recall (Sensitivity or True Positive Rate)
recall = recall_score(y_test, y_pred_rf)
```

```
print(f"Recall: {recall:.4f}")
```

Recall: 0.6350

```
[31]: #Random Forest
# Calculate F1-Score
f1 = f1_score(y_test, y_pred_rf)
print(f"F1-Score: {f1:.4f}")
```

F1-Score: 0.6840

```
[32]: # Calculate accuracy for different races and genders
accuracy_by_race = (y_pred_rf[race_groups == 1] == y_test[race_groups == 1]).
    →mean()
accuracy_by_sex = (y_pred_rf[sex_groups == 1] == y_test[sex_groups == 1]).mean()

print(f"Accuracy for White individuals: {accuracy_by_race:.2f}")
print(f"Accuracy for Male individuals: {accuracy_by_sex:.2f}")
```

Accuracy for White individuals: 0.85

Accuracy for Male individuals: 0.82

```
[33]: #Random Forest

accuracy_by_race_black = (y_pred_rf[race_black == 1] == y_test[race_black == 1]).
    →mean()

accuracy_by_sex_female = (y_pred_rf[sex_female] == y_test[sex_female]).mean()

print(f"Accuracy for Black individuals: {accuracy_by_race_black:.2f}")
print(f"Accuracy for Female individuals: {accuracy_by_sex_female:.2f}")
```

Accuracy for Black individuals: 0.91

Accuracy for Female individuals: 0.93

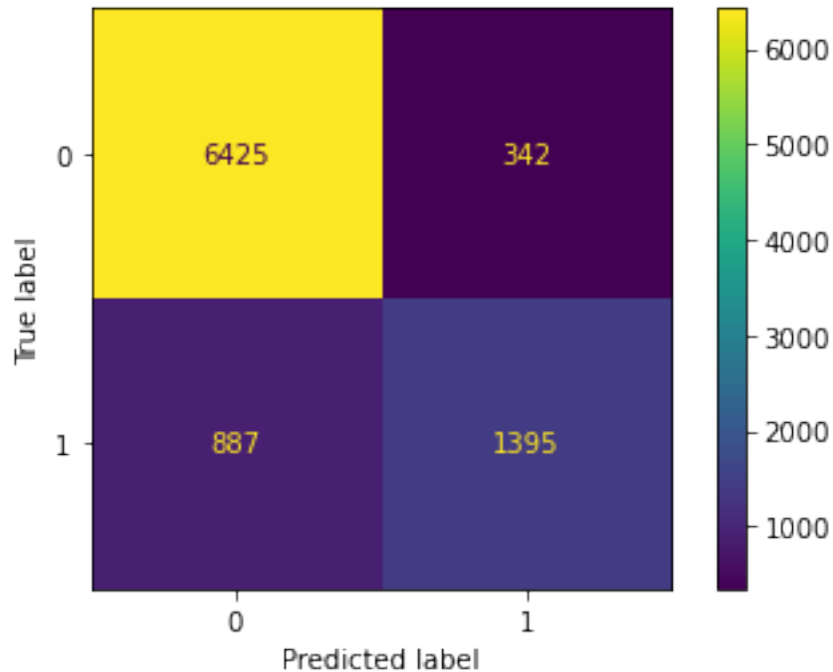
```
[34]: #Work completed for Random Forest, Now to perform same for Gradient Boosting
```

```
[35]: #Gradient Boosting
# Predict on the test set
y_pred_gb = gb_model.predict(X_test)

# Compute the confusion matrix and transpose it
cm2 = confusion_matrix(y_test, y_pred_gb)

# Display the confusion matrix with transposed axes
ConfusionMatrixDisplay(confusion_matrix=cm2).plot()
```

```
[35]: <sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x2c1c74d2fd0>
```

```
[36]: #Gradient Boosting

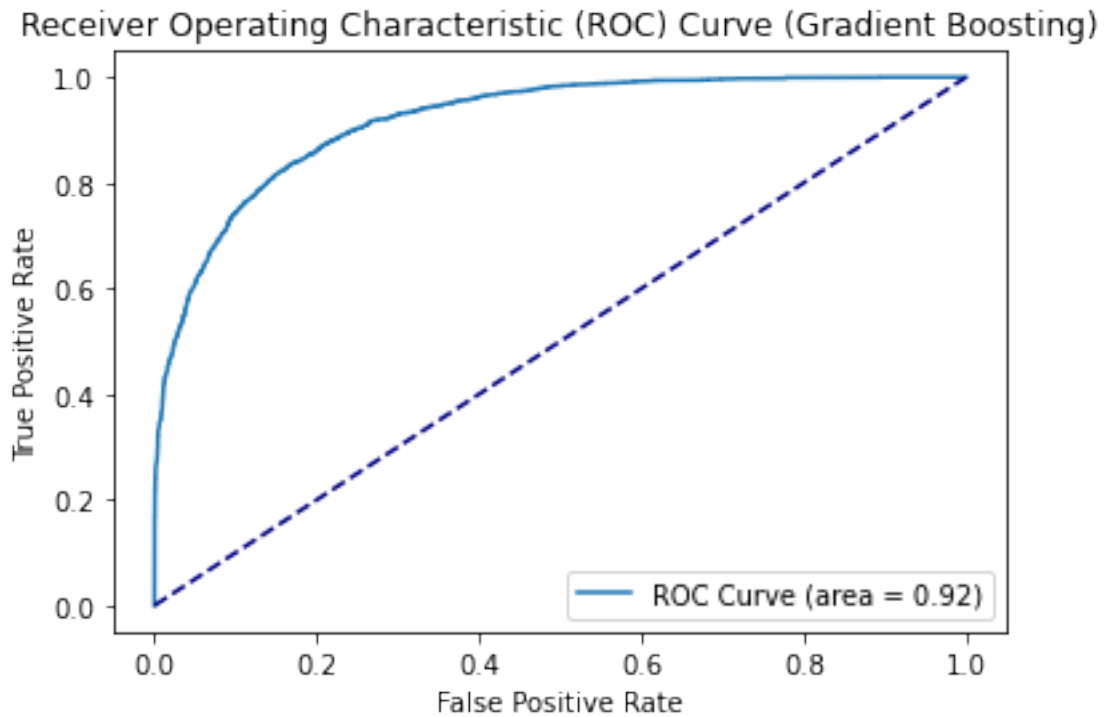
# Calculate the probability estimates for the test set
y_proba = gb_model.predict_proba(X_test)[:, 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

# Compute AUC
auc = roc_auc_score(y_test, y_proba)

# Plot ROC Curve
plt.figure()
plt.plot(fpr, tpr, label=f"ROC Curve (area = {auc:.2f})")
plt.plot([0, 1], [0, 1], color="navy", linestyle="--")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve (Gradient Boosting)')
plt.legend(loc="lower right")
plt.show()

# Print AUC score
print(f"AUC: {auc:.2f}")
```



AUC: 0.92

```
[37]: #Gradient Boosting

# Predict on the test set
y_pred = gb_model.predict(X_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred_gb)
print(f"Accuracy: {accuracy:.4f}")
```

Accuracy: 0.8642

```
[38]: #Gradient Boosting

# Calculate Precision
precision = precision_score(y_test, y_pred_gb)
print(f"Precision: {precision:.4f}")
```

Precision: 0.8031

```
[39]: #Gradient Boosting

# Calculate Recall (Sensitivity or True Positive Rate)
recall = recall_score(y_test, y_pred_gb)
```

```
print(f"Recall: {recall:.4f}")
```

Recall: 0.6113

```
[40]: #Gradient Boosting
```

```
# Calculate F1-Score
f1 = f1_score(y_test, y_pred_gb)
print(f"F1-Score: {f1:.4f}")
```

F1-Score: 0.6942

```
[41]: #Gradient Boosting
```

```
# Calculate accuracy for different races and genders
accuracy_by_race = (y_pred_gb[race_groups == 1] == y_test[race_groups == 1]).
    ↪mean()
accuracy_by_sex = (y_pred_gb[sex_groups == 1] == y_test[sex_groups == 1]).mean()

print(f"Accuracy for White individuals: {accuracy_by_race:.2f}")
print(f"Accuracy for Male individuals: {accuracy_by_sex:.2f}")
```

Accuracy for White individuals: 0.86

Accuracy for Male individuals: 0.83

```
[42]: #Gradient Boosting
```

```
accuracy_by_race_black = (y_pred_gb[race_black == 1] == y_test[race_black == 1]).
    ↪mean()

accuracy_by_sex_female = (y_pred_gb[sex_female] == y_test[sex_female]).mean()

print(f"Accuracy for Black individuals: {accuracy_by_race_black:.2f}")
print(f"Accuracy for Female individuals: {accuracy_by_sex_female:.2f}")
```

Accuracy for Black individuals: 0.92

Accuracy for Female individuals: 0.93

```
[ ]: #Based on this evaluation, decide which model you would choose and explain your
    ↪reasoning.
```

```
"""The output above can be compared across all three models to make a
    ↪determination as to which is the best.
```

```
At least by each metric measured.
```

```
Accuracy for the 3 models:
```

```
Logistic Regression: 78.51%
```

```
Random Forest: 85.20%
```

```
Gradient Boosting: 86.42%
```

From the comparison of the accuracy we can see that Gradient Boosting performed,
→ the best of the three models
when comparing on accuracy alone. To gain a more complete picture we can,
→ compare the other measures such
as precision and recall.

Precision for the 3 models:

Logistic Regression: 70.93%

Random Forest: 74.12%

Gradient Boosting: 80.31%

For the comparison of precision we see more of what was reflected in the,
→ measures of accuracy.

That is, logistic regression performed the worst out of the three models used,
→ Random Forest performed

better than logistic regression, and Gradient Boosting performed the best out,
→ of the 3. Interestingly the

gap between random forest and gradient boosting is much larger in the precision,
→ measure than it is in the
accuracy measure.

Recall for the 3 models:

Logistic Regression: 25.02%

Random Forest: 63.50%

Gradient Boosting: 61.13%

Here we see a slight change in the pattern between the models. Logistic,

→ Regression performed much much worse

than the other two models with a dismal 25.02% recall. Random Forest performed,
→ the best of the three models

beating out Gradient Boosting in this measure.

F1 score for the 3 models:

Logistic Regression: 36.99%

Random Forest: 68.40%

Gradient Boosting: 69.42%

For the F1 score we see a return to the pattern established with the previous,
→ scores where Logistic

Regression performed the worst, Random Forest performed better, and Gradient,
→ Boosting performed the best

when using this measurement.

AUC for the 3 models

Logistic Regression: 0.59

Random Forest: 0.90

Gradient Boosting: 0.92

So by using the standard measures available to be able to objectively compare

↳ the models Gradient Boosting

comes out as the best model in all measures except for Recall where Random

↳ Forest was able to slightly

outperform Gradient Boosting. So, using these metrics we would recommend the

↳ Gradient Boosting model.

But we need to check how the models perform with the data we are using. This

↳ can be accomplished by using

the additional measures performed using gender and race.

Logistic Regression:

Accuracy for White individuals: 77%

Accuracy for Male individuals: 74%

Accuracy for Black individuals: 88%

Accuracy for Female individuals: 89%

Random Forest:

Accuracy for White individuals: 85%

Accuracy for Male individuals: 82%

Accuracy for Black individuals: 91%

Accuracy for Female individuals: 93%

Gradient Boosting:

Accuracy for White individuals: 86%

Accuracy for Male individuals: 83%

Accuracy for Black individuals: 92%

Accuracy for Female individuals: 93%

Using these measures there is no clear reason not to remain with the Gradient

↳ Boosting Model.

It outperformed Logistic Regression in this measure as well as the previous

↳ measures, and performs

as well (accuracy for female individuals) as the random forest on some measures

↳ and better than random forest

on others. As such the recommendation would remain with the Gradient Boosting

↳ method.

""

[]: ""Although this dataset isn't classifying people into high-stakes categories,
 ↳like "offender" or "non-offender,"
 it is predicting whether someone earns more than \$50k a year. From a
 ↳responsible AI perspective,
 this can still be problematic. Explain how this type of prediction could
 ↳negatively impact the individuals
 being classified by the algorithm, as well as the broader group of stakeholders
 ↳who might not interact
 with the algorithm directly but could still be affected by its outcomes:

Though the stakes in the determination are not high without context, depending
 ↳on the context this can change.

For instance, the reason behind making this determination can have a wide range
 ↳of effects. If this information
 is used to determine the qualification for assistance of some type then there
 ↳are a number of individuals
 who could be incorrectly classified in the over 50k category. An instance such
 ↳as this can have far reaching
 effects such as homelessness, inability to obtain food, inability to pay for
 ↳care and more.

This would have far reaching economic impacts that would far exceed that of
 ↳just the individual affected.

Likewise if this is used for determining loan qualification then this type of
 ↳model can unjustly
 categorize an individual and make them unable to acquire the loan. As this
 ↳loan could be for a variety
 of reasons the effects again could be varied and far reaching.

Perhaps the most concerning issue with this dataset is the disparity between
 ↳measurements of each of the
 categories. For instance, the groups "white" and "male" make up the largest
 ↳percentage of the data, but
 they produce the lowest accuracy. This leads me to believe that there exists
 ↳the possibility that
 the model may not be accurately capturing other groups in the output. That is
 ↳to say, while the accuracy is
 higher in each model for "Black" or "Female" individuals, they also make up a
 ↳much smaller portion
 of the dataset, and this may be skewing the results, as aside from the number of
 ↳observations in the
 dataset there is not anything that greatly differentiates the groups in terms
 ↳of one having more information
 to make a better estimate than another group... To reiterate, the concern is
 ↳that the accuracy tends to be

higher for the groups with less data because there is less data, and that if
 → these groups were proportional
 to the largest group then accuracy of the model may be deprecated.
 It would be worth finding if this difference is statistically significant, and
 → if it is,
 then resolve this difference. When comparing across groups it is important to
 → keep things
 within the same scale, as such either the number of observations in the other
 → groups would need to be
 increased, or the sample scheme which is used to select customers needs to be
 → changed to a method
 such as stratified sampling in order to resolve this.
 Maybe this is less of an issue with this type of model, but for me, this is a
 → big red flag in
 any model which is supposed to provide measures of accuracy on classification
 → task.
 And perhaps this is more of an issue with regression than it is classification
 → and this red flag is a
 red herring.
 ""

[44]: *#Need to install LaTeX extension for jupyter, have it for R, not this*
 → *installation of python*
 !pip install nbconvert

Requirement already satisfied: nbconvert in c:\users\super\anaconda3\lib\site-packages (6.1.0)
 Requirement already satisfied: testpath in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (0.5.0)
 Requirement already satisfied: jupyter-core in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (4.8.1)
 Requirement already satisfied: nbformat>=4.4 in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (5.1.3)
 Requirement already satisfied: traitlets>=5.0 in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (5.1.0)
 Requirement already satisfied: defusedxml in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (0.7.1)
 Requirement already satisfied: jinja2>=2.4 in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (2.11.3)
 Requirement already satisfied: mistune<2,>=0.8.1 in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (0.8.4)
 Requirement already satisfied: entrypoints>=0.2.2 in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (0.3)
 Requirement already satisfied: pandocfilters>=1.4.1 in c:\users\super\anaconda3\lib\site-packages (from nbconvert) (1.4.3)
 Requirement already satisfied: bleach in c:\users\super\anaconda3\lib\site-

packages (from nbconvert) (4.0.0)
 Requirement already satisfied: nbclient<0.6.0,>=0.5.0 in
 c:\users\super\anaconda3\lib\site-packages (from nbconvert) (0.5.3)
 Requirement already satisfied: pygments>=2.4.1 in
 c:\users\super\anaconda3\lib\site-packages (from nbconvert) (2.10.0)
 Requirement already satisfied: jupyterlab-pygments in
 c:\users\super\anaconda3\lib\site-packages (from nbconvert) (0.1.2)
 Requirement already satisfied: MarkupSafe>=0.23 in
 c:\users\super\anaconda3\lib\site-packages (from jinja2>=2.4->nbconvert) (1.1.1)
 Requirement already satisfied: nest-asyncio in
 c:\users\super\anaconda3\lib\site-packages (from
 nbclient<0.6.0,>=0.5.0->nbconvert) (1.5.1)
 Requirement already satisfied: async-generator in
 c:\users\super\anaconda3\lib\site-packages (from
 nbclient<0.6.0,>=0.5.0->nbconvert) (1.10)
 Requirement already satisfied: jupyter-client>=6.1.5 in
 c:\users\super\anaconda3\lib\site-packages (from
 nbclient<0.6.0,>=0.5.0->nbconvert) (6.1.12)
 Requirement already satisfied: pyzmq>=13 in c:\users\super\anaconda3\lib\site-
 packages (from jupyter-client>=6.1.5->nbclient<0.6.0,>=0.5.0->nbconvert)
 (22.2.1)
 Requirement already satisfied: tornado>=4.1 in
 c:\users\super\anaconda3\lib\site-packages (from jupyter-
 client>=6.1.5->nbclient<0.6.0,>=0.5.0->nbconvert) (6.1)
 Requirement already satisfied: python-dateutil>=2.1 in
 c:\users\super\anaconda3\lib\site-packages (from jupyter-
 client>=6.1.5->nbclient<0.6.0,>=0.5.0->nbconvert) (2.8.2)
 Requirement already satisfied: pywin32>=1.0 in
 c:\users\super\anaconda3\lib\site-packages (from jupyter-core->nbconvert) (228)
 Requirement already satisfied: jsonschema!=2.5.0,>=2.4 in
 c:\users\super\anaconda3\lib\site-packages (from nbformat>=4.4->nbconvert)
 (3.2.0)
 Requirement already satisfied: ipython-genutils in
 c:\users\super\anaconda3\lib\site-packages (from nbformat>=4.4->nbconvert)
 (0.2.0)
 Requirement already satisfied: setuptools in c:\users\super\anaconda3\lib\site-
 packages (from jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert) (58.0.4)
 Requirement already satisfied: six>=1.11.0 in c:\users\super\anaconda3\lib\site-
 packages (from jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert) (1.16.0)
 Requirement already satisfied: attrs>=17.4.0 in
 c:\users\super\anaconda3\lib\site-packages (from
 jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert) (21.2.0)
 Requirement already satisfied: pyparsing>=2.0.2 in
 c:\users\super\anaconda3\lib\site-packages (from
 jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert) (2.4.6)
 Requirement already satisfied: webencodings in
 c:\users\super\anaconda3\lib\site-packages (from bleach->nbconvert) (0.5.1)
 Requirement already satisfied: packaging in c:\users\super\anaconda3\lib\site-

packages (from bleach->nbconvert) (23.2)

```
[45]: !pip install nbconvert[webpdf]
```

```
Requirement already satisfied: nbconvert[webpdf] in
c:\users\super\anaconda3\lib\site-packages (6.1.0)
Requirement already satisfied: pygments>=2.4.1 in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (2.10.0)
Requirement already satisfied: traitlets>=5.0 in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (5.1.0)
Requirement already satisfied: bleach in c:\users\super\anaconda3\lib\site-
packages (from nbconvert[webpdf]) (4.0.0)
Requirement already satisfied: defusedxml in c:\users\super\anaconda3\lib\site-
packages (from nbconvert[webpdf]) (0.7.1)
Requirement already satisfied: nbclient<0.6.0,>=0.5.0 in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (0.5.3)
Requirement already satisfied: testpath in c:\users\super\anaconda3\lib\site-
packages (from nbconvert[webpdf]) (0.5.0)
Requirement already satisfied: mistune<2,>=0.8.1 in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (0.8.4)
Requirement already satisfied: jinja2>=2.4 in c:\users\super\anaconda3\lib\site-
packages (from nbconvert[webpdf]) (2.11.3)
Requirement already satisfied: jupyter-core in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (4.8.1)
Requirement already satisfied: jupyterlab-pygments in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (0.1.2)
Requirement already satisfied: nbformat>=4.4 in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (5.1.3)
Requirement already satisfied: entrypoints>=0.2.2 in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (0.3)
Requirement already satisfied: pandocfilters>=1.4.1 in
c:\users\super\anaconda3\lib\site-packages (from nbconvert[webpdf]) (1.4.3)
Collecting pyppeteer==0.2.2
  Downloading pyppeteer-0.2.2-py3-none-any.whl (145 kB)
Requirement already satisfied: urllib3<2.0.0,>=1.25.8 in
c:\users\super\anaconda3\lib\site-packages (from
pyppeteer==0.2.2->nbconvert[webpdf]) (1.26.7)
Requirement already satisfied: appdirs<2.0.0,>=1.4.3 in
c:\users\super\anaconda3\lib\site-packages (from
pyppeteer==0.2.2->nbconvert[webpdf]) (1.4.4)
Requirement already satisfied: tqdm<5.0.0,>=4.42.1 in
c:\users\super\anaconda3\lib\site-packages (from
pyppeteer==0.2.2->nbconvert[webpdf]) (4.62.3)
Collecting websockets<9.0,>=8.1
  Downloading websockets-8.1.tar.gz (58 kB)
Collecting pyee<8.0.0,>=7.0.1
  Downloading pyee-7.0.4-py2.py3-none-any.whl (12 kB)
Requirement already satisfied: MarkupSafe>=0.23 in
```

c:\users\super\anaconda3\lib\site-packages (from jinja2>=2.4->nbconvert[webpdf]) (1.1.1)

Requirement already satisfied: nest-asyncio in c:\users\super\anaconda3\lib\site-packages (from nbclient<0.6.0,>=0.5.0->nbconvert[webpdf]) (1.5.1)

Requirement already satisfied: async-generator in c:\users\super\anaconda3\lib\site-packages (from nbclient<0.6.0,>=0.5.0->nbconvert[webpdf]) (1.10)

Requirement already satisfied: jupyter-client>=6.1.5 in c:\users\super\anaconda3\lib\site-packages (from nbclient<0.6.0,>=0.5.0->nbconvert[webpdf]) (6.1.12)

Requirement already satisfied: pyzmq>=13 in c:\users\super\anaconda3\lib\site-packages (from jupyter-client>=6.1.5->nbclient<0.6.0,>=0.5.0->nbconvert[webpdf]) (22.2.1)

Requirement already satisfied: python-dateutil>=2.1 in c:\users\super\anaconda3\lib\site-packages (from jupyter-client>=6.1.5->nbclient<0.6.0,>=0.5.0->nbconvert[webpdf]) (2.8.2)

Requirement already satisfied: tornado>=4.1 in c:\users\super\anaconda3\lib\site-packages (from jupyter-client>=6.1.5->nbclient<0.6.0,>=0.5.0->nbconvert[webpdf]) (6.1)

Requirement already satisfied: pywin32>=1.0 in c:\users\super\anaconda3\lib\site-packages (from jupyter-core->nbconvert[webpdf]) (228)

Requirement already satisfied: jsonschema!=2.5.0,>=2.4 in c:\users\super\anaconda3\lib\site-packages (from nbformat>=4.4->nbconvert[webpdf]) (3.2.0)

Requirement already satisfied: ipython-genutils in c:\users\super\anaconda3\lib\site-packages (from nbformat>=4.4->nbconvert[webpdf]) (0.2.0)

Requirement already satisfied: attrs>=17.4.0 in c:\users\super\anaconda3\lib\site-packages (from jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert[webpdf]) (21.2.0)

Requirement already satisfied: six>=1.11.0 in c:\users\super\anaconda3\lib\site-packages (from jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert[webpdf]) (1.16.0)

Requirement already satisfied: setuptools in c:\users\super\anaconda3\lib\site-packages (from jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert[webpdf]) (58.0.4)

Requirement already satisfied: pyrsistent>=0.14.0 in c:\users\super\anaconda3\lib\site-packages (from jsonschema!=2.5.0,>=2.4->nbformat>=4.4->nbconvert[webpdf]) (0.18.0)

Requirement already satisfied: colorama in c:\users\super\anaconda3\lib\site-packages (from tqdm<5.0.0,>=4.42.1->pyppeteer==0.2.2->nbconvert[webpdf]) (0.4.4)

Requirement already satisfied: packaging in c:\users\super\anaconda3\lib\site-packages (from bleach->nbconvert[webpdf]) (23.2)

Requirement already satisfied: webencodings in c:\users\super\anaconda3\lib\site-packages (from bleach->nbconvert[webpdf]) (0.5.1)

```

Building wheels for collected packages: websockets
  Building wheel for websockets (setup.py): started
  Building wheel for websockets (setup.py): finished with status 'done'
  Created wheel for websockets: filename=websockets-8.1-cp39-cp39-win_amd64.whl
  size=62758
  sha256=15b237d9c31ae2b5b6af45b22e42545071dba7f836dfb9b60da7a3d40416be92
  Stored in directory: c:\users\super\appdata\local\pip\cache\wheels\d8\b9\ao\b9
  7b211aeda2ebd6ac2e43fc300d308dbf1f9df520ed390cae
Successfully built websockets
Installing collected packages: websockets, pyee, pyppeteer
Successfully installed pyee-7.0.4 pyppeteer-0.2.2 websockets-8.1

```

```
[48]: !pip install pyppeteer
```

```

Requirement already satisfied: pyppeteer in c:\users\super\anaconda3\lib\site-
packages (0.2.2)
Requirement already satisfied: appdirs<2.0.0,>=1.4.3 in
c:\users\super\anaconda3\lib\site-packages (from pyppeteer) (1.4.4)
Requirement already satisfied: tqdm<5.0.0,>=4.42.1 in
c:\users\super\anaconda3\lib\site-packages (from pyppeteer) (4.62.3)
Requirement already satisfied: pyee<8.0.0,>=7.0.1 in
c:\users\super\anaconda3\lib\site-packages (from pyppeteer) (7.0.4)
Requirement already satisfied: urllib3<2.0.0,>=1.25.8 in
c:\users\super\anaconda3\lib\site-packages (from pyppeteer) (1.26.7)
Requirement already satisfied: websockets<9.0,>=8.1 in
c:\users\super\anaconda3\lib\site-packages (from pyppeteer) (8.1)
Requirement already satisfied: colorama in c:\users\super\anaconda3\lib\site-
packages (from tqdm<5.0.0,>=4.42.1->pyppeteer) (0.4.4)

```

```
[49]: from pyppeteer import launch
```

```

# Launch with chromium download allowance
browser = await launch(headless=True, args=["--allow-chromium-download"])
await browser.close()

```

```

[W:pyppeteer.chromium_downloader] start chromium download.
Download may take a few minutes.
100%|          | 136913619/136913619 [00:03<00:00, 37426557.38it/s]
[W:pyppeteer.chromium_downloader]
chromium download done.
[W:pyppeteer.chromium_downloader] chromium extracted to:
C:\Users\super\AppData\Local\pyppeteer\pyppeteer\local-chromium\588429

```

```
[ ]: \end{document}
```